

reely.

a content studio that lives in your chat. send an idea, a raw clip, or a youtube link — get back a captioned vertical short. hermes writes the words, ffmpeg cuts the video. no capcut, no prompts, no converter sites.

MODE 1 – GENERATE

topic → viral reel

hermes brainstorms a viral angle, scripts + voices it, returns a 14–20s reel

MODE 2 – EDIT

footage → reel

send a clip, get it transcribed, captioned, cut, sticker'd

MODE 3 – CLIP

youtube → reel

paste a link + timestamps, get a captioned short of that moment

One engine, three doorways

Every mode collapses to the same core: Hermes generates the words, ffmpeg burns them onto video. That's why three modes is one build, not three.

Two problems kill posting: not knowing *what* to make (the blank page), and hating the edit (CapCut to trim, one site to convert, another to caption, another to sticker). Reely solves both in one chat — it can **brainstorm a viral concept and hand you the finished reel**, or take your clip/link and cut one. You edit any step by just replying.

MODE 1 · GENERATE

"give me a viral reel on why upi won." → Hermes **brainstorms 2–3 viral angles**, you pick one → writes the hook + script → ElevenLabs voiceover → captions over kinetic text/b-roll → a tight **14–20s reel** built for the algorithm.

MODE 2 · EDIT

Send raw footage → Whisper transcribes → Hermes writes captions/meme text → ffmpeg trims, captions, formats → subtitled clip or sticker pack.

MODE 3 · CLIP (YOUTUBE)

Paste a link + "2:30–3:15" (or a topic) → yt-dlp grabs **only that segment** → captioned vertical short of the moment.

The wow in the demo: **from "I don't know what to post" to a finished 14–20s reel in one message**. Then show the range — paste a 2-hour podcast link, ask for one moment (Mode 3), and a captioned clip comes back the same 40 seconds later. Three inputs, one chat window.

Why it plays to Hermes's strengths

Hermes is strong at **content generation** (scripting, captions, meme copy) *and* **tool use** (driving ffmpeg/yt-dlp/Whisper). Reely uses both at once and registers as a native Hermes **skill** — the agent reads your message, routes to the right operation chain, and returns the file. That is textbook "Way 2" eligibility: a real product running on Hermes with end users interacting through it.

SCOPE DISCIPLINE (WHY THIS IS ONE 8-HOUR BUILD, NOT FIVE)

All three modes reuse the caption engine, the ffmpeg op library, the chat gateway, the job queue, and the watermark. Mode 1 adds one TTS call. Modes 2 & 3 add one transcription call. **No Remotion** — kinetic captions are pure ffmpeg. That is the entire delta.

Name: "reely" is a working title. Alternates to check for domains: cliply, yapclip, cutaway, snip.chat. Pick before launch so the watermark + share loop carry a real brand.

How it's built

ffmpeg-first. Hermes is the brain and the eligibility. Everything heavy is capped and queued.

Hermes · skill registration + NL routing + scripting (mandatory)

ffmpeg · all trim/caption/convert/assemble

yt-dlp · segment download (Mode 3)

Whisper · transcription (Modes 2 & 3)

ElevenLabs · voiceover (Mode 1, +25)

Linkup · live facts to ground reels (Mode 1, +25)

Convex · jobs/users/usage (+25)

Cloudflare · hosting + file delivery (+25)

Dodo · ₹99 watermark unlock (+25)

Sponsor power-ups — how each earns its +25 (and the proof to show)

TOOL	REAL WORK IN REELY	PROOF FOR THE MENTOR
ElevenLabs	Mode 1 voiceover — the reel's narration IS the product, not a dead snippet	live demo: generate a reel, it speaks
Linkup	Mode 1 grounds the script in live web facts + sources before writing — timely, not hallucinated	code (src/search) + a live query returning facts that appear in the reel
Convex	main backend — jobs, users, per-day usage (free-tier cap), Pro state	repo + Convex dashboard rows updating live
Dodo	live ₹99 checkout that flips the user to Pro (drops watermark) via webhook	Dodo dashboard + a real checkout on stage
Cloudflare	hosts the landing/waitlist + serves output files (Pages/R2/Workers)	live URL + CF dashboard
Wispr Flow	dictate 500+ words during the event (build notes / the pitch) — a personal action, no product change	Wispr stats screenshot

All six = +150 on top of the base rubric. Only Linkup needed new product code (the grounding step); the other four are already core to reely; Wispr is you dictating on the day.

WHERE HERMES IS USED (THE ELIGIBILITY ANSWER)

The rule: use Hermes to **build** the agent (Way 1) **or** have the product **run on** Hermes with users interacting through it (Way 2). **reely does both — and Way 2 is the strong one: reely literally is a Hermes skill.** Hermes is the runtime users talk to, not a library we call once.

HERMES CAPABILITY	WHAT IT DOES IN REELY
Gateway	Telegram/WhatsApp/Discord — how the user sends the idea/clip/link and gets the reel back. The product surface is the Hermes gateway.
Skill	The media toolset registers as a native Hermes skill; the agent auto-routes a message to the right operation chain.
Content gen	Brainstorms viral angles, writes the script, writes captions/meme text (Mode 1 + Mode 2/3 captions).
Subagents	Optional: parallel ops (transcribe while fetching b-roll) fanned out at runtime.
Memory	Optional: remembers a user's voice pick / caption style across requests.
Way 1 (build)	We also code the whole thing <i>with</i> Hermes — keep prompt-history + commits as receipts for mentors.

Proof to show a mentor: the live Telegram bot doing a real edit (Way 2 capability doing real work) + the Hermes session receipts (Way 1).

Data flow (shared core)



The NL → operation schema (the security firewall)

User text **never** becomes a shell string. Hermes maps intent to a fixed, validated operation list; each op is executed with argument arrays, not interpolated commands. This is the single most important design choice — it kills command injection and makes chaining reliable.

```
[ { op:"clip", start:"2:30", end:"3:15" },  
  { op:"captions", source:"whisper", style:"karaoke" },  
  { op:"format", aspect:"9:16" },  
  { op:"watermark", text:"made with reely", show:!isPro } ]
```

Convex schema (core)

```
jobs { id, userId, mode, source, ops[], status, inBytes, outBytes, ms, isPro }  
users { id, platform, handle, freeUsedToday, isPro, refCode }  
usage { userId, day, count } // enforces the free-tier daily cap
```

Why no Remotion

Remotion renders React in headless Chromium — 2–3GB per render, your stated crash risk. Every caption style you need (word-by-word karaoke, meme top/bottom text, kinetic titles) is achievable with ffmpeg drawtext and .ass subtitles at a fraction of the cost. Keep Remotion out of the buildathon entirely; revisit only if you later want templated motion graphics, and then run it as a single-concurrency isolated worker.

So it never crashes under load

Your #1 concern, answered. A capped queue with hard per-job limits — not fire-and-forget.

QUEUE & CONCURRENCY

- Global concurrency = **CPU cores – 1** (e.g. 3 ffmpeg jobs at once)
- Per-user cap = **1 active job**; the rest queue ("you're #4 in line")
- Queue depth > ~20 → reject politely, don't accumulate an OOM

PER-JOB HARD LIMITS (REJECT BEFORE PROCESSING)

- File ≤ **50MB** (Telegram bot-API limit anyway)
- Duration ≤ **5 min**, resolution ≤ **1080p** (downscale 4K on ingest)
- Wall-clock timeout **60–120s** → kill the whole process group
- `ffmpeg -threads 2` so one job can't grab all cores

YOUTUBE INGEST – THE KEY INFRA WIN

Never download the whole 2-hour video. `yt-dlp --download-sections "*2:30–3:15"` fetches **only the requested segment**. Cap segment length (≤3 min), cap source duration, and reject live streams. This keeps Mode 3 as light as Mode 2.

DISK HYGIENE (THE SILENT KILLER)

Unique temp dir per job; delete input + output **immediately after send**. Disk-space guard refuses new jobs below a free-space threshold. TTL sweep for orphaned files. On timeout, `kill -9` the process group or zombie `ffmpeg/yt-dlp` procs eat RAM.

ROUGH CAPACITY ON ONE MODEST BOX (4 VCPU / 8GB)

3 concurrent `ffmpeg` jobs ≈ 0.6–1.2GB RAM, each caption/trim job finishes in seconds to ~30s. At ~3 jobs/min throughput that's **thousands of shorts/day** queued smoothly. A viral spike doesn't crash it — it queues, and "you're #6 in line" is acceptable UX. What would crash it (Remotion, whole-video downloads, uncapped file sizes, no timeouts) is exactly what the rules above remove.

Contrast with `died.fyi`: there, virality directly multiplied browser-farm cost. Here, the caps + segment-only downloads + `ffmpeg`-first keep marginal cost per short at cents, and the queue absorbs spikes. This is the light shape you wanted.

Hosting & cost — one VPS is enough

Hermes is **the** Telegram/WhatsApp backend — you don't build a bot server. Its gateway connects to chat, routes to the reely skill, and sends the file back. Hermes runs on the same VPS as the `ffmpeg` workers. **The model is an external OpenAI-compatible API call, so no GPU is needed** — that's what keeps hosting cheap.

WHAT IT COSTS

ITEM	NOTE	ROUGH
Hermes software	open-source, just a process	free
VPS (4 vCPU/8GB)	Hermes + <code>ffmpeg</code> workers	\$15–48/mo
LLM tokens	brainstorm/script/route	\$200 credits
ElevenLabs + Whisper	voice + transcribe (API)	power-up + cents

Buildathon gives \$200 OpenAI credits; hourly VPS cost for the event is pennies.

SINGLE-VPS BUDGET (4 VCPU / 8GB)

- Hermes process ~0.5–1GB (orchestrator; LLM is remote)
- 3 concurrent `ffmpeg` jobs ~0.6–1.2GB behind the queue
- `yt-dlp` + Whisper-via-API ~ near-zero local footprint

Scales by adding worker boxes *behind the same queue* later — no rearchitcting. Telegram first (robust); WhatsApp second (QR-session bridge, flakier). What breaks single-VPS — self-hosted Whisper/LLM (GPU) and Remotion — is already ruled out.

What will break, and the handling

The list judges and reality will probe. Handle the top rows; acknowledge the rest.

Command & chaining

CASE	HANDLING
Vague command ("make it better")	Ask one clarifying question; never guess silently
Impossible chain ("to mp3 + add watermark")	Detect at schema stage, explain why
Mid-chain failure (step 2 of 3 dies)	Report which step; return last good artifact
Timestamp ambiguity / out of range	Normalize "30"→0:30; clamp to duration; reject start>end

YouTube / yt-dlp (Mode 3 specific)

CASE	HANDLING
Private / age-restricted / members-only / geo-blocked	Catch yt-dlp error, return a clear "can't access this video" message
Live stream / premiere / very long video	Reject live; cap source duration; segment-only download
YouTube bot-detection / rate limit	Backoff + retry; optional cookies; queue so you don't hammer
Requested moment beyond video length	Clamp or ask; don't fetch empty range
Copyright / ToS	Ownership/fair-use checkbox; attribution on output; position as clipping your own / licensed content, not a rip tool

Media & platform

CASE	HANDLING
Fake extension, corrupt, zero-byte	ffprobe validate first; reject cleanly
Rotation metadata / odd dimensions	–autorotate; scale to even width/height
No audio (asked for audio) / no video (asked to trim)	Detect streams; explain the mismatch
Output exceeds platform send limit	Auto-compress-to-fit or return a download link
WhatsApp 16MB / Discord 25MB / Telegram 50MB	Know each; target output under the active platform's cap

Security / abuse (do not skip)

Command injection via NL	Fixed op schema + arg arrays; never shell-interpolate user text
SSRF / "edit this URL" tricks	ffmpeg –protocol_whitelist file; no network protocols
Decompression bombs / crafted files	Resource caps + timeout + sandboxed worker contain blast radius
Spam (100 jobs)	Per-user rate limit + queue cap already handle it

Winning the track + the ₹99 unlock

Signups are 25x. A private tool has no free loop — so we engineer three, and one feature is both the ad and the revenue.

LOOP 1 · WATERMARK = AD + REVENUE

Free outputs carry "made with reely". Every reel someone posts is a distributed ad. Removing it is the paid unlock — the single most proven micro-payment pattern (Epik, upscalers all monetized exactly this).

LOOP 2 · STICKERS IN-CHAT

Sticker/GIF output spreads through WhatsApp & Telegram groups by design — the one output that propagates *inside* the app it's built on.

LOOP 3 · BUILD-IN-PUBLIC + SIGNUP GATE

The "I replaced CapCut + 4 sites with one chat" post is strong builder-Twitter content. Pair it with "3 free shorts, then sign up to continue" — that turns reach into the 25x meaningful-action metric.

The Dodo model

FREE – THE ACQUISITION ENGINE

₹0

3 shorts/day · sign up to continue

- All 3 modes
- Watermark + ref code on output
- 50MB / 5-min / 1080p caps
- Standard queue

PRO – ONE-TIME

₹99 / \$1.99

Dodo · UPI one-tap · no login wall

- **No watermark**, HD export
- Bigger files, longer clips, batch/chained ops
- **Priority queue** (skip the line)
- Voice picks + caption styles

One-time, not subscription (impulse psychology, no churn in a one-day event). Watermark-removal + priority-queue are the two things people impulse-pay for at "this is actually useful." Every ₹99 is also a Revenue-track cross-bonus.

HONEST TRACK NOTE

This scores naturally high in "**AI as Agency**" (working product shipping real output = 20x) and **Revenue**. In **Virality** it competes only via the three loops above. If the room lets you, weigh Agency — but the plan here assumes Virality as you chose.

Build timeline (10 AM → 5:30 PM demo)

Ship one mode end-to-end first, then add the other two on the shared engine. Proof in the back half.

- **10:00 – Kickoff**
Lock Virality track + idea. Split: 1 on Hermes skill + NL schema, 1 on the ffmpeg op library + queue, 1 on gateway + Cloudflare + launch posts.
- **10:05 – before any code**
Launch tweet ("building reely...") + one-field waitlist on Cloudflare Pages. Install Datafast with mentor read-only access.
- **10:30–1:00 – Mode 2 first (edit)**
It's the simplest full loop: upload → Whisper → captions → ffmpeg burn → send. One working captioned clip beats three half-modes.
- **1:00–2:00 – Mode 3 (YouTube)**
Add yt-dlp --download-sections as a new ingest in front of the same caption engine. Reuses everything downstream.
- **2:00–3:00 – Mode 1 (generate)**
Hermes brainstorms viral angles → picks one → script → ElevenLabs voice → captions over kinetic text → 14–20s reel. Reuses the caption + assemble path.
- **2:30 (parallel) – proof**
Post real outputs. Get peers on the floor to run their clips → each is a signup + a reshare (3× amplification).
- **3:00–4:00 – Dodo + queue caps**
₹99 watermark unlock live (1 real test purchase). Concurrency cap, per-job limits, disk cleanup, timeouts.
- **4:00–4:45 – polish + guardrails**
Ownership checkbox for YouTube, rate limits, output validation, milestone post #2.
- **4:45–5:15 – proof pack**
Screens: Convex users/jobs table, Datafast visitors, Dodo checkout, X analytics, notable reshares.
- **5:30 – demo**
20s context → run all 3 modes live (paste a judge's fav YouTube link for Mode 3) → 1 min proof numbers → Q&A. Submit before deadline.

CUT-LINE IF BEHIND

Ship Mode 2 + Mode 3 (both edit-shaped, share the engine) and demo those. Mode 1 (generate) is the first thing to drop — it adds TTS + b-roll risk. Two solid modes > three shaky ones.

Launch-day distribution

One product, native framing per channel. All copy in @PsudoKit voice — lowercase, specific, no corporate.

SLOT 1 · ~10:05

launch

building out loud

building reely at a buildathon rn.

don't know what to post? it brainstorms a viral reel and makes it. already got a clip or a youtube link? it captions and cuts it. all in one chat. no capcut, no prompts.

link soon.

image: 10s screen rec — "make me a viral reel on X" → finished 15s short · self-reply: waitlist link here (not main tweet)

SLOT 2 · ~1 PM

proof

specific number

it works. fed it a 2 hour podcast link and "clip 41:20 to 41:55", got a captioned reel back in 40 seconds.

never scrubbing a timeline for one quote again.

image: the actual output reel · self-reply: "did the same with a raw phone clip. auto captions, straight to sticker"

SLOT 3 · ~6:30 PM

the drill (their post format)

relatable pain

you know the drill.

trim a clip → capcut. convert to mp3 → some site. caption it → another. make a sticker → another. minutes of uploading and downloading.

built reely so it's all one chat. free to try 📌

image: before/after — 5 app icons vs one chat bubble · self-reply: direct link + "reply a youtube link and i'll clip it live"

Other channels

CHANNEL	NATIVE FRAMING
r/SideProject	[Launch] "chat bot that turns a youtube link or raw clip into a captioned short" — first line = the tab-switching pain, live link, GIF demo
r/InternetIsBeautiful	Frame as a toy: "paste a youtube link + a timestamp, get a captioned vertical clip back". No signup wall visible
Instagram / Reels	15s screen-rec: "i made this reel by sending one chat message". Reel made BY the tool = the proof. Link in bio
Discord (fire FIRST)	Builder/college servers before public: "send me a youtube link, i'll clip any moment into a captioned short". Early velocity = algo boost

Reply-clipping is the cheat code: people reply a YouTube link + a moment, you post the finished short back. Each is a signup, a public post, and live social proof. Block the evening for it.

The checklist

If it's on screen at 5:30, it scores. If it isn't, it doesn't exist.

BUILD

- ☐ Hermes skill registered, NL → op schema
- ☐ Mode 2 (edit) end-to-end captioned clip
- ☐ Mode 3 (YouTube) segment-only download
- ☐ Mode 1 (generate) brainstorm → 14–20s voiced reel
- ☐ Watermark on free + ref code
- ☐ Dodo ₹99 unlock live (1 real purchase)
- ☐ Queue cap + per-job limits + cleanup
- ☐ Ownership checkbox (YouTube)

PROOF PACK

- ☐ Convex users + jobs table (live)
- ☐ Datafast visitors, mentor read-only
- ☐ Dodo dashboard, real checkout
- ☐ X analytics: impressions > followers
- ☐ Notable reshares (open profiles live)
- ☐ A gallery of real outputs made on the floor

DISTRIBUTION FIRED

- ☐ X launch + 3 slot posts
- ☐ Discord servers (before public)
- ☐ r/SideProject + r/InternetIsBeautiful
- ☐ IG reel (made by the tool)
- ☐ Reply-clipped 15+ links live

NUMBERS TO BEAT

- **~300 signups** = 100 pts + overflow (25×)
- **1,000+ visitors** = L5 (10×)
- **5 power-ups live** = +125
- **Real Dodo revenue** = Revenue cross-bonus
- **1 notable reshare** = L4 amplification

"ffmpeg-first. cap everything. the reel it makes
is the ad. the watermark is the price."